

pkg sentinel: AI 시대 소프트웨어 공급망 악성 패키지 탐지 엔진 및 Agentic Capability Manifest

7팀 캡스톤 최종 보고서 · 2026-05-31

1. 프로젝트 개요

1.1 배경

대규모 언어 모델(LLM)이 코드 생성·실행에 광범위하게 사용되면서 소프트웨어 공급망에 두 가지 새로운 위협 표면이 형성되었다.

(1) **슬롭스쿼팅(slopsquatting)**. LLM은 실제로 존재하지 않는 패키지명을 추천하는 환각(hallucination)을 일으킨다. 공격자가 이 환각 패키지명을 레지스트리(PyPI·npm)에 미리 등록(선점)하면, AI 추천을 신뢰한 개발자가 악성 패키지를 설치하게 된다. 레지스트리는 업로드 시 코드 리뷰를 수행하지 않으므로, 공격자 등록 시점부터 레지스트리 측 삭제 시점까지의 시간 창(time window)이 공격 기회가 된다.

(2) **Agentic 패키지·스킬**. Model Context Protocol(MCP) 서버와 AI 에이전트 스킬(SKILL.md 기반)이 급증하면서, 파일·네트워크·셸 권한을 보유한 패키지가 늘어났다. 이 영역에서는 악성 로직이 실행 코드가 아니라 AI 에이전트에게 주어지는 자연어 지시문에 위치하는 새로운 공격 형태가 등장했다.

1.2 목적

본 프로젝트는 레지스트리에 **등록되어 있으나 악의적인** 패키지를, 설치·실행 없이 정적으로 탐지하는 엔진을 구현하고, 실제 악성 패키지 데이터셋 및 표준 비교 도구를 사용해 성능을 측정하는 것을 목적으로 한다. 또한 패키지·스킬 제작자가 필요 권한을 사전 선언하고 도구가 실제 탐지 행위와 대조하는 **Agentic Capability Manifest** 표준을 제안한다.

1.3 위협 모델

LLM 환각/agentic 패키지 → 공격자 레지스트리 등록 → AI 추천 신뢰 설치 → 악성코드 실행
|
pkg sentinel가 메우는 구간
(등록 시점 ~ 삭제 시점 time window)

탐지 대상: 자격증명 탈취, 원격 코드 실행(RCE), 설치 훅(install-hook) 기반 송출, 난독화 페이로드, 버전 간 악성 코드 주입, 스킬 지시문 인젝션.

2. 시스템 구조

2.1 분석 원칙

- 패키지 아카이브를 디스크에 설치하지 않고 **메모리로 스트리밍**하여 핵심 파일만 추출·분석한다.

- 판정은 추출된 증거(Evidence)와 단계 실행 결과만으로 결정하며, 패키지의 등록일·인기도·다운로드 수는 판정에 사용하지 않는다.
- 필수 단계(행위 시퀀스, TTP 매칭, LLM 검증) 중 하나라도 실패하면 부분 판정을 내리지 않고 **ERROR**로 종료한다.

2.2 7계층 / 21단계 파이프라인

탐지 로직은 7계층(L0~L6)으로 구성되며, 실제 실행은 21개 단계(stage_00~stage_20)로 순차 수행된다. 계층 구조의 설계 의도는, 비용이 낮고 결정적인 검사(레지스트리 조회·DB 매칭)를 앞단에 두어 초기에 판정을 단축하고, 비용이 높은 분석(임베딩 매칭·LLM 검증)을 뒤로 배치하는 것이다. 각 단계는 산출한 증거(Evidence)와 실행 결과(StageResult)를 다음 단계로 누적 전달하며, 최종 판정(stage_20 직후)에서 누적된 증거 전체를 근거로 판정을 결정한다.

단계는 역할에 따라 세 종류로 구분된다. **(a) 게이트 단계**는 조건 충족 시 후속 분석을 생략하고 즉시 판정을 확정한다(stage_00 미등록, stage_01 known-malicious, stage_07 agentic 악성). **(b) 필수 단계**(stage_08·11·16)는 판정의 근거가 되며, 하나라도 실패하면 부분 판정 없이 **ERROR**로 종료한다. **(c) 참고 단계**(stage_03·04·20)는 메타데이터를 수집해 리포트에 포함하지만 판정값에는 직접 반영하지 않는다 — 판정이 패키지의 평판·인기도·출처 신뢰도가 아니라 코드 행위에만 근거하도록 분리한 것이다.

계층	단계	기능	구분
L0 레지스트리·위협 인텔	stage_00 레지스트리 확인	PyPI/npm 등록 여부 및 버전·아카이브 URL 조회. 미등록 시 CANNOT_ANALYZE 로 종료(소스 없음).	게이트
	stage_01 Threat Filter	암호화 DB(SQLCipher, 220k+ OSV/GHSA + OSSF malicious-packages)에서 known-malicious / popular / typosquat 매칭. exact match 시 즉시 MALICIOUS .	게이트
	stage_02 공격 이력	OSV/GHSA 기준 이름·버전 과거 사고 조회. 현재 버전이 영향 범위 밖이면 historical로 분리.	보조
	stage_03 OpenSSF Scorecard	저장소 URL로 OpenSSF Scorecard 조회(브랜치 보호·코드 리뷰·의존성 갱신·바이너리 아티팩트 등 항목별 점수). 리포트에 첨부하며 판정 미반영.	참고
L1 소스 추출·Agentic 게이트	stage_04 SLSA 프로비던스	메타데이터로 SLSA 레벨(빌드 출처 증명·서명 존재 여부)을 추정. 리포트에 첨부하며 판정 미반영.	참고
	stage_05 캐시 조회	아카이브 해시 기반 6-트리거 무효화로 이전 분석 결과 재사용.	보조
	stage_06 전 파일 소스 추출	아카이브를 메모리로 스트리밍하여 소스·바이너리 파일 분리 추출. 후속 전 단계의 입력 기반.	필수 기반
L2 행위 시퀀스	stage_07 Agentic 분류	MCP 서버·AI 스킴 패키지 여부 판별. agentic이고 악성이면 게이트로 판정 단축, 그 외엔 증거만 보강.	게이트
	stage_08 Behavior Sequence	Python AST + JavaScript tree-sitter로 API 호출을 추출하고, 민감정보 읽기·인코딩/난독화·동적 코드 실행·원격 전송 4개 공격 차원으로 분류.	필수
	stage_09 문자열 상수 풀	base64/hex 인코딩 문자열, 고엔트로피(난독화) 문자열 탐지.	보조
	stage_10 버전 diff	이전 버전 대비 신규 추가된 위험 API 추출(예: event-stream-xz 류 버전 주입).	보조

계층	단계	기능	구분
	stage_11 TTP 매칭	MITRE ATT&CK TTP 임베딩 + 규칙 기반 매칭(상세는 2.2.1).	필수
	stage_12 카테고리 이상	패키지 설명(description)과 실제 탐지 행위의 불일치 탐지.	보조
	stage_13 악성 지표 매치	악성 지표 분류 체계(49 항목, 활성 매치 36개; install-hook-exec/eval-credential theft-shell execution 등)와 매칭, 파일 단위 risk-combo 승격.	보조
	stage_14 Sequential pattern mining	호출 시퀀스의 순차 패턴 마이닝.	보조
	stage_15 Taint slicing	source(env/파일/비밀)→sink(http/exec) 데이터 흐름만 추출해 stage_16 입력 토큰 절감.	보조
L4 LLM 이중 검증	stage_16 LLM 검증	Claude 멀티 에이전트 3종(의미·버전 diff·의존성)이 각자 판정 후 합의 투표. 정상 패키지의 악신호를 강등.	필수
L5 추가 분석 (옵션)	stage_17 의존성 재귀	직접/개발 의존성을 재귀 분석(--deps 플래그).	옵션
	stage_18 바이너리	.so/.dll/.node/.pyd 등 바이너리 파일 분석.	옵션
	stage_19 샌드박스	격리 환경 동적 실행 관찰(--sandbox 플래그).	옵션
L6 판정·표준 출력	stage_20 NIST SSDF	NIST SSDF 준수 체크(참고). 이후 누적 증거로 최종 판정 결정 및 표준 포맷 리포트 구성.	참고/판정

위 단계 중 **게이트 단계**는 조기 종료를 통해 명백한 케이스(미등록·기지 악성·agentic 악성)에서 불필요한 연산을 생략한다. **L3 패턴 매칭 계층**은 단일 기법에 의존하지 않고 문자열 분석·버전 diff·TTP 매칭·지표 매치·시퀀스 마이닝·테인트 분석을 병렬적으로 적용하여 서로 다른 각도에서 증거를 수집한다. **L4 LLM 검증**은 L2-L3가 수집한 증거를 문맥적으로 재평가하여, 정상 패키지에서 발생한 약한 신호를 강등하고 실제 악성 신호를 확정하는 정밀도 조정 역할을 한다.

성능 측정 시에는 L0의 위협 인텔 조회(stage_01)를 비활성화하여, DB 매칭이 아닌 순수 행위 기반(L2-L4) 분석 성능을 측정하였다.

2.2.1 MITRE ATT&CK TTP 매칭 (stage_11)

TTP(Tactics, Techniques, and Procedures)는 공격자의 행위 패턴을 분류한 체계로, MITRE ATT&CK이 업계 표준 지식베이스를 제공한다. 본 단계는 패키지의 코드 행위를 ATT&CK 기법과 의미 수준에서 대조하여, 정확한 문자열·시그니처가 일치하지 않더라도 의미가 유사한 변종 행위를 탐지하는 것을 목표로 한다. 처리 과정은 stage_08이 추출한 각 파일의 행위 시퀀스(FileSequence) 단위로 다음과 같이 수행된다.

- 시퀀스 → 자연어 변환.** 행위 시퀀스를 임베딩 모델이 처리할 수 있는 문장으로 변환한다. 4개 공격 차원(민감정보 읽기·인코딩/난독화·동적 코드 실행·원격 전송)의 요약과 API 호출 체인(`call_a -> call_b -> ...`)을 결합한 영문 서술을 생성한다.
- 임베딩.** 변환된 문장을 Sentence-BERT 계열 문장 임베딩 모델(all-MiniLM-L6-v2)로 벡터화한다.

3. **Top-K 검색.** MITRE ATT&CK 기법을 동일 모델로 사전 임베딩해 둔 로컬 인덱스 (`mitre_attack_embedded.json`)에서 코사인 유사도 기준 상위 K개(기본 K=3) 후보를 검색한다.
4. **임계값 적용.** 유사도에 따라 신호 강도를 구분한다. - 0.50 미만: 잡음으로 간주하여 무시. - 0.70 이상: 후보 매칭으로 채택(STRONG_THRESHOLD). - 0.85 이상: 강한 매칭으로 간주하여 `HIGH_RISK` / `MALICIOUS` 분기 조건에 사용(VERY_STRONG_THRESHOLD).
5. **규칙 기반 보강.** 임베딩이 놓칠 수 있는 결정적 패턴을 규칙 매칭(`apply_rules`)으로 보강한다. 규칙 적용은 결정적 매칭이므로 유사도 1.0으로 기록되어 임베딩 매칭과 함께 후속 단계로 전달된다.

이 단계의 매칭 결과(파일 경로, 시퀀스, 매칭된 TTP, 유사도)는 stage_16 LLM 검증의 입력으로 사용되며, 강한 매칭(유사도 ≥ 0.85 , severity $\neq$ LOW)은 LLM 판정과 무관하게 위험 분기로 진입한다. 반대로 약한 매칭(유사도 0.70–0.85, severity LOW)에 대해 LLM이 BENIGN으로 판정하면 해당 신호는 승격되지 않는다(2.3 참조).

2.3 판정 규칙

판정	조건
MALICIOUS	high-severity TTP ≥ 1 AND LLM=malicious AND 최강 악성 증거 confidence ≥ 0.85
HIGH_RISK	(TTP match ≥ 1 OR version_diff critical) AND LLM $\in$ {suspicious, malicious}
SUSPICIOUS	weak TTP(LLM $\neq$ BENIGN) OR version_diff any OR LLM-only suspicious(confidence ≥ 0.5 , quorum 2)
CLEAN	전 단계 통과 AND 증거 없음, 또는 weak TTP + LLM=BENIGN
ERROR	필수 단계(08/11/16) 중 1개 이상 실패
CANNOT_ANALYZE	stage_00 미등록 확정

LLM의 BENIGN 판정은 약한 TTP 신호(유사도 0.70–0.85, severity LOW)를 덮어쓴다. 강한 매칭(유사도 ≥ 0.85 , severity $\neq$ LOW)은 LLM 판정과 무관하게 위험 분기로 진입한다.

2.4 Agentic Capability Manifest

제작자가 필요 권한(파일·네트워크·셸·환경변수 등)을 매니페스트에 선언하면, 엔진이 stage_07–16에서 실제 탐지한 행위(capability)와 대조하여 미선언 권한을 식별한다. 도구별로 상이한 권한 표현(self 27종, MITRE 28종)은 15개 표준 capability 어휘(filesystem-read/write, shell, network, env-secrets, code-exec, credential-paths, db-access, mcp-client/server, llm-call, tool-loop, memory-persistent, dynamic-tool-load, agent-to-agent)로 정규화하여 선언과 탐지를 동일 어휘로 비교한다.

2.5 표준 출력 포맷

CycloneDX 1.5 VEX, STIX 2.1 / TAXII 2.1, HMAC 서명 웹훅, Falco rules · Tetragon TracingPolicy, SafeDep pmg policy.

3. 실험

3.1 데이터셋

DataDog malicious-software-packages-dataset (2026-05 기준): - 악성 아카이브 약 22,052개 / 고유 패키지 약 11,704개. - 구성: npm malicious_intent 20,588개 + compromised_lib 1,260개 + ai-skills 204개; PyPI malicious_intent 1,806 패키지(2,483 아카이브); ide_extensions 카테고리(신설). - 사고 발견 (post-discovery) 후 수집된 스냅샷이며 파일명 날짜는 발견일을 의미한다.

악성 샘플링: 고정 시드(seed=42) 기반 무작위 추출로 재현 가능하며, 10MB 초과 아카이브는 제외하였다.

정상 샘플: 현재 레지스트리의 인기 패키지를 사용하였다(npm: react, vue, angular 등 / PyPI: requests, urllib3, fastapi 등).

3.2 LLM 환각률 측정

- 모델: gpt-5.2, claude-opus-4-8. 총 1,000회 질의.
- 추출된 패키지명을 PyPI-npm 레지스트리 API로 검증.

지표	값
raw 환각률 (레지스트리 미등록 이름 비율)	6.65%
검증 후 완전 허구 비율	3.13% (A등급 1.10%)
반복성 (동일 질의 5회 중 3회 이상 동일 환각)	32.6%

3.3 npm 비교 (n = 273)

동일 분모(Socket가 분석 가능한 273개)에서 비교.

도구	recall	precision	F1
pkgsentinel (행위 기반, L0 OFF)	0.652	1.000	—
pkgsentinel (L0 ON)	0.912	—	—
pkgsentinel (프로덕션 경로)	—	—	0.974
Socket.dev	0.937	—	0.962

3.4 PyPI 비교 (n = 139)

도구	recall	precision	F1
pkgsentinel (claude)	0.886	0.972	0.927
pkgsentinel (stub)	0.886	0.843	—
Bandit	0.487	0.470	—
Semgrep (Docker)	0.212	0.298	—
OSV	0.76 (사후 등재)	—	—

행위 기반 단독 시 정상 패키지 오탐 45건이 발생하였으며, LLM 검증 단계 적용 후 오탐은 2건으로 감소하였다.

3.5 Zero-day 다중 사건 (DB 비등록 부분집합)

3개의 독립 명명 캠페인을 대상으로, OSV 등재 이전 상태를 가정하여 측정.

사건	pkgsentinel	OSV	njsscan
Shai-Hulud (2025-09)	100%	0%	0%
Mini Shai-Hulud (2026-05)	100%	0%	0%
TrapDoor (2026-05)	78% (페이로드 없는 decoy 6건 제외 시 90%)	0%	0%

OSV는 사건 공개 전에는 정의상 0%이다. TrapDoor 분석 과정에서 package.json install-hook을 통한 네트워크 송출 미탐을 확인하여 해당 탐지 규칙(`_NPM_SCRIPT_DANGEROUS`)을 구현하였고, 탐지율이 61%에서 78%로 상승하였다(정상 패키지 오탐 0).

3.6 ai-skills (n = 204, 동일 분모)

도구	recall
OSV-Scanner	0.000
Semgrep	0.098
Bandit	0.142
pkgsentinel	0.152
pkgsentinel + markdown-pinpoint	0.284

ai-skills 204개 중 158개(77%)는 Python 코드를 포함하지 않으며, 악성 로직이 SKILL.md의 자연어 지시문에 위치한다. 지시문 분석 모듈(markdown-pinpoint)을 추가 구현하여 탐지율이 0.152에서 0.284로 상승하였다(정상 89개 대상 오탐 0).

3.7 측정 도구

도구	제공	분류
Socket.dev	Socket Inc	행위 기반 공급망 분석
OSV-Scanner	Google	DB(advisory) 매칭
Bandit	PyCQA	Python 정적 분석(SAST)
Semgrep	Semgrep Inc	다언어 정적 분석(SAST)
njsscan	MobSF / OpenSecurity	Node.js 정적 분석(SAST)

4. 구현 환경

- 언어: Python 3.11
- 정적 분석: Python `ast`, tree-sitter(JavaScript)
- 임베딩: sentence-transformers (all-MiniLM-L6-v2)
- LLM: Anthropic Claude API (검증 에이전트 기본 claude-sonnet-4-5), OpenAI/Anthropic API(환각 실험)
- 데이터베이스: SQLCipher (AES-256)

- 배포: Docker, docker-compose

5. 참고문헌

선행 연구

1. J. Spracklen et al., *We Have a Package for You: A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs*, USENIX Security Symposium, 2024.
2. A. P. Felt, E. Chin, S. Hanna, D. Song, D. Wagner, *Android Permissions Demystified* (Stowaway), ACM Conference on Computer and Communications Security (CCS), 2011.
3. *Malware2ATT&CK: 행위 기반 ATT&CK 기법 매핑*, Computers & Security (ScienceDirect S0167404824000737), 2024.
4. *Unveiling Malicious Logic in Open-Source Software Packages*, arXiv:2512.12559, 2025.
5. N. Reimers, I. Gurevych, *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, EMNLP, 2019.

표준·프레임워크

6. MITRE ATT&CK. <https://attack.mitre.org>
7. Open Source Vulnerabilities (OSV), Google. <https://osv.dev>
8. S. Brown, *The C4 model for visualising software architecture*. <https://c4model.com>
9. Model Context Protocol (MCP), Anthropic.
10. OWASP Top 10 for LLM Applications / OWASP MCP Top 10.
11. NIST Secure Software Development Framework (SSDF).
12. CycloneDX 1.5 (VEX), STIX 2.1 / TAXII 2.1, SLSA, OpenSSF Scorecard.

비교·분석 도구

13. Socket.dev. <https://socket.dev>
14. OSV-Scanner, Google. <https://github.com/google/osv-scanner>
15. Bandit, PyCQA. <https://github.com/PyCQA/bandit>
16. Semgrep, Semgrep Inc. <https://semgrep.dev>
17. njsscan, MobSF / OpenSecurity. <https://github.com/ajinabraham/njsscan>
18. skill-scanner, Sentry (getsentry/skills) — 프롬프트 인젝션 패턴 참조.

데이터셋·선례

19. DataDog/malicious-software-packages-dataset. <https://github.com/DataDog/malicious-software-packages-dataset>
20. DataDog/guarddog. <https://github.com/DataDog/guarddog>
21. K. Gilbertson, npm 권한 모델 제안, 2018.
22. Deno Permissions, Google Capslock — 권한 어휘 참조.

검증 대상 사건

23. Shai-Hulud (npm 자기복제 웜), 2025-09.

24. Mini Shai-Hulud (TeamPCP), 2026-05. (Orca Security, The Hacker News 보도)

25. TrapDoor (암호화폐 자격증명 탈취), 2026-05. (The Hacker News 보도)